

# Audio Deepfake Detection API

## System Architecture Document

Version 1.0 | Full Technical Deep-Dive

Generated: 2026-06-23 08:04 UTC

Audience: Engineering, MLOps & Security Teams

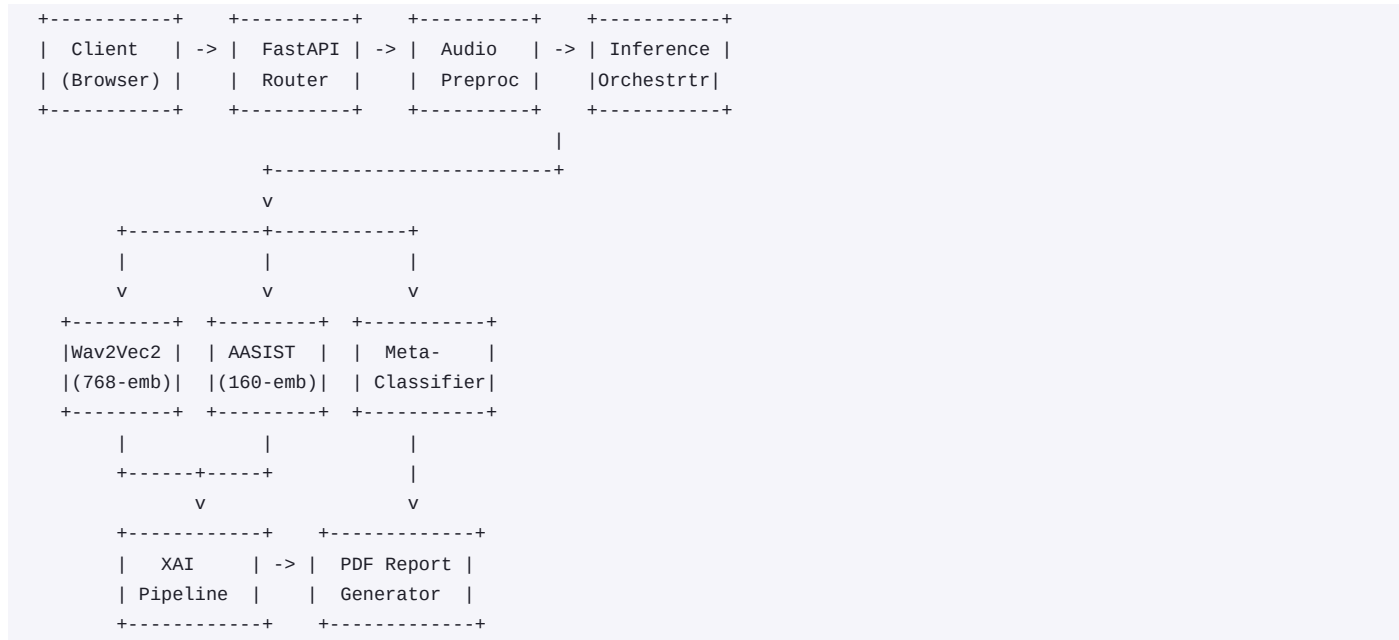
Repository: [https://github.com/MLSDepK/AudioResearch\\_and\\_training/](#)

# 1. System Overview

The Audio Deepfake Detection API is a production-grade REST microservice that detects whether an audio sample contains authentic human speech (bona-fide) or AI-generated deepfake / spoof audio. It employs a three-model ensemble: Wav2Vec2 (HuggingFace transformer), AASIST (graph-based anti-spoofing), and a custom dual-branch MLP meta-classifier that fuses embeddings from both sub-models. Every request automatically generates XAI attention maps and a downloadable PDF forensic report.

## High-Level Architecture Flow

Figure 1: End-to-End Request Flow



## Key Design Decisions

- CPU-only inference -- no GPU dependency. All three models run on CPU for broader deployment compatibility.
- Stdin/stdout ffmpeg pipe for non-WAV codecs -- eliminates temp file race conditions and supports MP3, M4A, OGG on-the-fly.
- Dual-branch Meta-Classifer -- prevents the 768-dim W2V2 embedding from numerically dominating the 160-dim AASIST embedding in a single concatenated layer.
- Always-on XAI + Report -- every detection call generates attention maps and a PDF report automatically (no query params needed).
- In-memory report cache -- PDF bytes stored in a dict keyed by UUID request\_id (lost on restart).

## 2. Audio Ingestion & Preprocessing

### 2.1 File Upload Handling

The API receives audio files via multipart/form-data upload (field name: 'file'). Supported extensions: .wav, .flac, .mp3, .m4a, .ogg. Maximum file size is 50 MB, maximum duration is 300 seconds (5 minutes).

### 2.2 Decoding Pipeline

Figure 2: Audio Ingestion Pipeline

```
Uploaded Bytes
|
v
sf.read() -- try soundfile first (WAV/FLAC)
| (fail?)
v
ffmpeg stdin pipe: ffmpeg -i pipe:0 -ar 16000 -ac 1 -f wav pipe:1
|
v
+ librosa.resample if sample rate != 16000
+ Peak normalization (divide by max abs)
+ Truncate to max_audio_samples (16000 * 300 = 4,800,000)
|
v
np.float32 array, shape (n_samples,)
```

#### Preprocessing Pseudocode

```
# key function: app/utils/audio_utils.py
async def load_audio_from_bytes(data, target_sr=16000):
    try:
        audio, sr = sf.read(io.BytesIO(data))    # WAV/FLAC
    except:
        audio = _convert_with_ffmpeg(data, 16000) # MP3/M4A/OGG via pipe
    if audio.ndim > 1: audio = audio.mean(axis=1) # mono
    if sr != 16000: audio = librosa.resample(...) # resample
    return audio.astype(np.float32), 16000

async def preprocess_pipeline(audio, sr):
    if sr != 16000: audio = librosa.resample(...)
    peak = max(abs(audio))
    if peak > 0: audio /= peak # normalize
    if len(audio) > 4800000: audio = audio[:4800000] # truncate
    return audio.astype(np.float32)
```

### 2.3 ffmpeg Format Support

When soundfile (libsndfile) cannot read a file (e.g., MP3, M4A, OGG), the system falls back to ffmpeg via a stdin-to-stdout pipe. The subprocess reads raw bytes from stdin, converts to 16 kHz mono WAV, and writes PCM data to stdout. This avoids writing temp files to disk, eliminating race conditions and cleanup overhead. Availability is checked once at startup via `check_ffmpeg()`.

### 3. Wav2Vec2 Transformer Pipeline

#### 3.1 Architecture

Wav2Vec2 is a self-supervised speech representation model from Facebook AI / HuggingFace. This project uses the fine-tuned checkpoint Vansh180/deepfake-audio-wav2vec2 which was fine-tuned on the ASVspoof 2019 Logical Access dataset for binary anti-spoofing classification (bona-fide vs spoof).

#### 3.2 Key Details

|                         |                                             |
|-------------------------|---------------------------------------------|
| Model ID:               | Vansh180/deepfake-audio-wav2vec2            |
| Source:                 | HuggingFace Hub (auto-downloaded)           |
| Hidden dim:             | 768                                         |
| Attention heads:        | 12                                          |
| Transformer layers:     | 12                                          |
| Output head:            | Linear(768 -> 2) with Softmax               |
| Feature encoder stride: | 320 samples (20ms at 16kHz)                 |
| Attention impl:         | eager (required for output_attentions=True) |

The model processes raw audio waveforms. The feature encoder convolves the audio into a sequence of 768-dim feature vectors at a 20ms frame rate. These vectors pass through 12 transformer layers with self-attention. The final hidden states are mean-pooled across the time dimension to produce a single 768-dim utterance-level embedding.

#### 3.3 Inference Modes

The Wav2Vec2Pipeline (app/models/wav2vec2\_pipeline.py) exposes three inference methods:

- infer():** Returns (confidence, logits\_vector). Uses only the classification head output (no hidden states). Fastest path.
- infer\_with\_embedding():** Returns (confidence, mean\_pooled\_hidden). Also requests hidden\_states[-1] from the transformer and mean across time to get a 768-dim embedding. Used for meta-classifier fusion.
- get\_attentions():** Returns (confidence, mean\_pooled\_hidden, list\_of\_attention\_maps). Enables output\_attentions=True, returning a list of 12 attention matrices each of shape (12 heads, seq\_len, seq\_len). Used for XAI.

Given raw audio  $x$ , the feature encoder produces a sequence of frame vectors  $F = \text{encoder}(x)$ ,  $F \in \mathbb{R}^{T \times 768}$ . Each transformer layer  $l$  applies multi-head self-attention:  $A_l = \text{softmax}(Q_l K_l^T / \sqrt{d_k})$ , then  $O_l = A_l V_l$ . After 12 layers, we extract  $H = \text{hidden\_states}[-1] \in \mathbb{R}^{T \times 768}$ , then  $\text{pooled} = \text{mean}(H, \text{dim}=0) \in \mathbb{R}^{768}$ . The classification head produces  $\text{logits} = W * \text{pooled} + b \in \mathbb{R}^2$ , and  $\text{confidence} = \text{softmax}(\text{logits})[1]$  (spoof class probability).

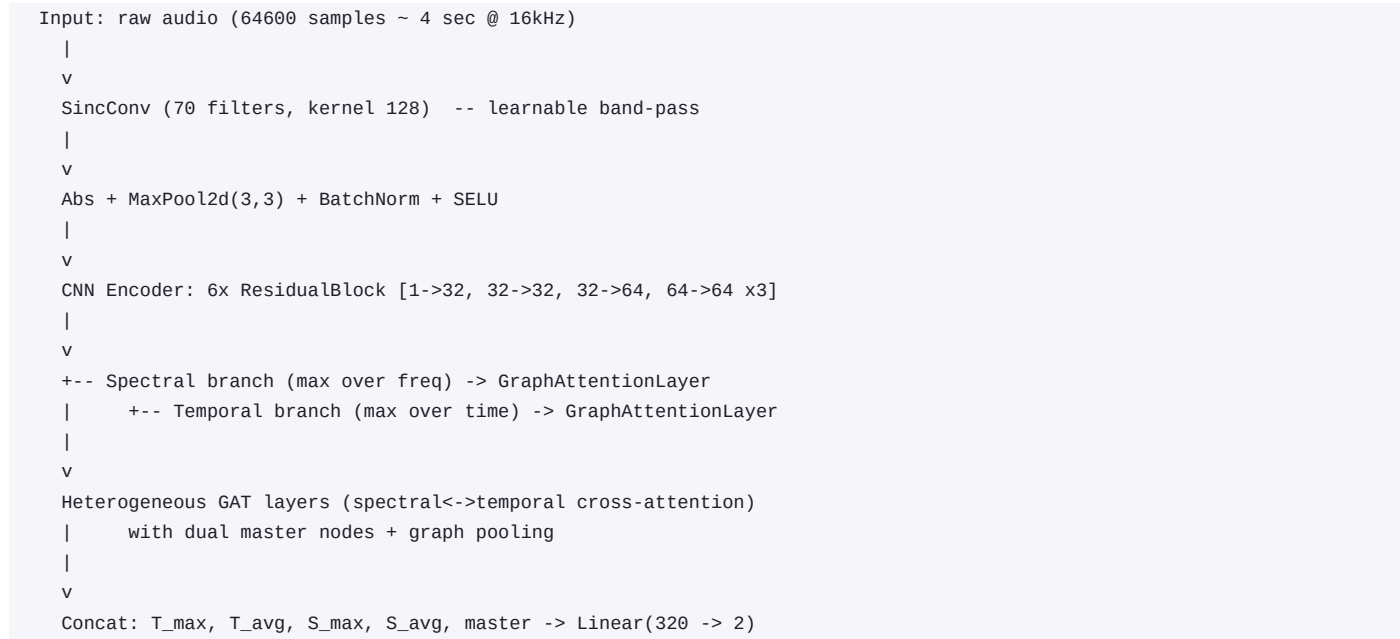
## 4. AASIST Anti-Spoofing Model

### 4.1 Architecture Overview

AASIST (Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks) is a state-of-the-art model for detecting fake audio. It processes raw waveforms through a Sinc-conv layer, CNN encoder, and a heterogeneous graph attention mechanism that jointly models spectral and temporal relationships.

### 4.2 Model Structure

Figure 3: AASIST Architecture



### 4.3 Layer-by-Layer Breakdown

**SincConv:** A learnable sinc-based band-pass filter bank applied directly to the raw waveform. It replaces traditional Mel-filterbanks with parameterized filters that can adapt during training. The kernel size is 128 samples (8 ms at 16 kHz). 70 filters produce 70 time-frequency channels.

**ResidualBlock:** Standard CNN residual blocks with 2D convolutions (kernel 2x3), SELU activation, batch norm, and max pooling (1x3). The first block maps 1->32 channels, subsequent blocks progress 32->32, 32->64, 64->64. Six blocks total.

**Graph Attention Layers:** After the CNN encoder, the feature map is split into spectral and temporal views. Each view passes through a GraphAttentionLayer that computes a learned self-attention over nodes (frames for temporal view, frequency bins for spectral view), then propagates information via attention-weighted aggregation.

**Heterogeneous GAT:** Four HtrgGraphAttentionLayer modules perform cross-attention between spectral and temporal branches using two separate master node tracks. This allows the model to jointly reason about time-frequency relationships. Graph pooling (top-k) reduces dimensionality at each stage.

The final embedding is a concatenation of 5 vectors: T\_max, T\_avg (max and average pooled temporal graph), S\_max, S\_avg (max and average pooled spectral graph), and the master node. This yields a 160-dim embedding (5 \* 32 = 160). The classification head maps 160 -> 2 for spoof probability.

### 4.4 Loading and Weight Details

|                |                                              |
|----------------|----------------------------------------------|
| Weights file:  | <a href="#">weights/aasist_backbone.pt</a>   |
| Source:        | <a href="#">Clova AI official AASIST.pth</a> |
| Input length:  | 64600 samples (~4s)                          |
| Embedding dim: | 160                                          |

**strict=False:**            Load ignores missing keys (backbone only)

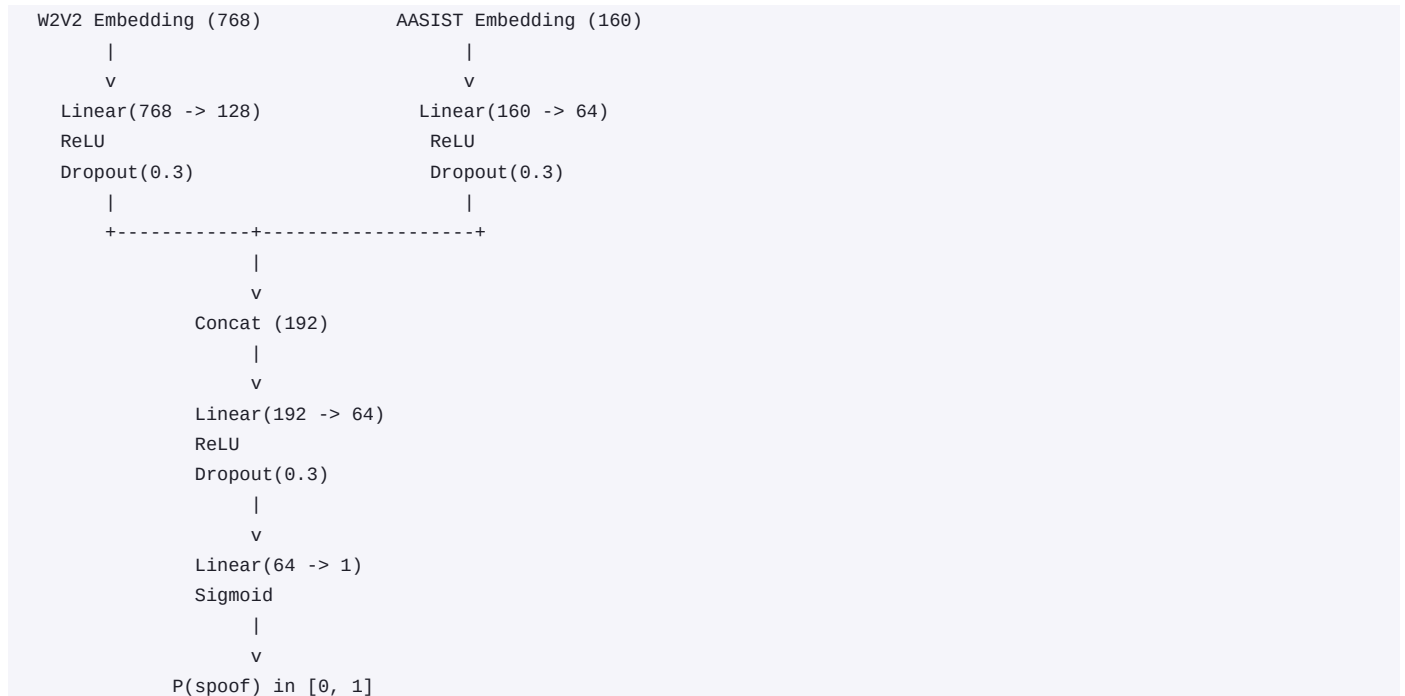
## 5. Dual-Branch Meta-Classifer

### 5.1 Motivation

The raw embeddings from Wav2Vec2 (768-dim) and AASIST (160-dim) operate in very different feature spaces. Simply concatenating them into a single 928-dim vector and feeding it to an MLP risks having the 768-dim W2V2 embedding numerically dominate the gradient signal. The dual-branch design processes each embedding through an independent learned projection before fusion.

### 5.2 Network Architecture

Figure 4: Dual-Branch MetaMLP Architecture



### 5.3 Forward Pass Details

#### PyTorch Implementation

```

class MetaMLP(nn.Module):
    def __init__(self, w2v_dim=768, aasist_dim=160, joint_dim=64):
        self.w2v_branch = nn.Sequential(
            Linear(768, 128), ReLU, Dropout(0.3))
        self.aasist_branch = nn.Sequential(
            Linear(160, 64), ReLU, Dropout(0.3))
        self.classifier = nn.Sequential(
            Linear(192, 64), ReLU, Dropout(0.3),
            Linear(64, 1), Sigmoid())

    def forward(self, x):
        # x.shape: (batch, 928) = cat([w2v_768, aasist_160], dim=-1)
        w2v = x[:, :768] # split
        aas = x[:, 768:]
        w_out = self.w2v_branch(w2v) # 768->128
        a_out = self.aasist_branch(aas) # 160->64
        fused = torch.cat([w_out, a_out], dim=-1) # 192
        return self.classifier(fused) # 192->64->1->sigmoid
  
```

### 5.4 Training Details

The MetaMLP was trained on ASVspoof 2019 Logical Access (LA) dataset. Training was performed in a Kaggle notebook

(research\_and\_training/kaggle\_mlp\_trainer.py). The training process:

Extract W2V2 and AASIST embeddings offline for each training sample

Train the dual-branch MLP with BCEWithLogitsLoss

Optimizer: AdamW (lr=1e-4, weight\_decay=1e-5)

Batch size: 64, validation split: 20%

Final validation accuracy: 99.17%

**Weights file:** [weights/meta\\_mlp.pt](#) (488 KB)

**Input dim:** 928 (768 + 160)

**Output:** scalar in [0,1] -- P(spoof)

## 5.5 Fallback Strategy

If the meta-classifier weights file is missing or fails to load (architecture mismatch), the system gracefully falls back to simple score averaging:  $P_{\text{ensemble}} = (P_{\text{w2v}} + P_{\text{aasist}}) / 2$ . This is logged as a warning instruction to retrain.



## 6. Inference Orchestrator

### 6.1 Orchestration Flow

The InferenceOrchestrator (app/inference/orchestrator.py) coordinates the entire detection pipeline. It is constructed with references to all three models and exposes a single async detect() method.

Figure 5: Orchestrator Sequence Diagram

```

detect(audio_bytes, xai_enabled=True)
|
v
1. Preprocess audio bytes -> numpy array (16kHz mono)
|
v
2. Wav2Vec2.infer_with_embedding(audio)
   -> (w2v_conf, w2v_emb_768)
|
v
3. AASIST.infer_with_embedding(audio)
   -> (aasist_conf, aasist_emb_160)
| (AASIST failure is caught & logged; continues with 0.5 fallback)
v
4. MetaClassifier.predict_proba(w2v_emb, aasist_emb, ...)
   -> ensemble_conf
|
v
5. DetectionResult with prediction, confidence, scores, latency
|
v
(if xai_enabled) -> _detect_with_xai()
|
+--> Wav2Vec2.get attentions() -> 12 raw attention matrices
+--> process_attentions()      -> Attention Rollout
+--> map_attention_to_timeline() -> per-frame scores
+--> generate_heatmap()        -> base64 PNG
+--> XaiResult(detection, attention_maps)

```

### 6.2 Code Structure

#### Orchestrator Core Logic

```

class InferenceOrchestrator:
    def __init__(self, wav2vec2, aasist, meta_classifier):
        self._w2v = wav2vec2
        self._aasist = aasist
        self._meta = meta_classifier

    async def detect(self, audio_bytes, xai_enabled=True):
        start = time.perf_counter()
        audio = await preprocess_audio_bytes(audio_bytes)
        w2v_conf, w2v_emb = await self._w2v.infer_with_embedding(audio)
        try:
            aasist_conf, aasist_emb = await self._aasist.infer_with_embedding(audio)
        except Exception:
            aasist_conf, aasist_emb = 0.5, None
        ensemble_conf = await self._meta.predict_proba(w2v_emb, aasist_emb,
                                                       w2v_conf, aasist_conf)

        det = DetectionResult(
            prediction="spoof" if ensemble_conf > 0.5 else "bona-fide",
            confidence=ensemble_conf,
            scores={"wav2vec2": w2v_conf, "aasist": aasist_conf, "ensemble": ensemble_conf},
            process_time_ms=round((time.perf_counter()-start)*1000, 1),

```

```
)  
if xai_enabled:  
    return await self._detect_with_xai(audio, det, start)  
return det
```

## 7. Explainable AI (XAI) Pipeline

### 7.1 Overview

The XAI pipeline provides interpretability for Wav2Vec2's decision by analyzing the self-attention matrices across all 12 transformer layers. It uses Attention Rollout to propagate attention through the residual connections, producing a per-frame importance score that indicates which temporal regions the model focused on during inference.

### 7.2 Attention Rollout Algorithm

Raw attention matrices from individual transformer layers only show direct attention patterns and ignore the residual connections (skip connections) that carry information around each self-attention block. Attention Rollout corrects this by mixing each layer's attention with the identity matrix:

For each layer  $l$  with attention matrix  $A_l$  (averaged over heads):

1. Mix with identity:  $A'_l = 0.5 * A_l + 0.5 * I$
2. Row-normalize:  $A'_l[i,:] = A'_l[i,:] / \text{sum}(A'_l[i,:])$
3. Accumulate:  $\text{rollout} = A'_l @ \text{rollout}$  (starting from  $I$ )

The final rollout matrix captures the total influence flow from input to output across the entire transformer stack. The per-frame importance is  $\text{rollout.mean}(\text{axis}=0)$  -- the average amount of attention each input frame receives from all output positions.

#### Attention Rollout Implementation

```
def _attention_rollout(attention):
    seq_len = attention[0].shape[-1]
    rollout = np.eye(seq_len)          # start with identity
    for layer_attn in attention:
        A = layer_attn.mean(axis=0)     # average 12 heads -> (T, T)
        A = 0.5 * A + 0.5 * np.eye(seq_len) # add residual connection
        A = A / A.sum(axis=-1, keepdims=True) # row-normalize
        rollout = A @ rollout           # accumulate
    return rollout                      # (T, T)

def process_attentions(attention):
    layer_scores = []
    for attn in attention:              # per-layer source-aggregated
        layer_scores.append(attn.mean(axis=0).mean(axis=0))
    rollout = _attention_rollout(attention)
    importance = rollout.mean(axis=0)    # per-frame importance
    layer_scores.append(importance)
    return layer_scores                # last element is rollout
```

### 7.3 Timeline Mapping

The per-frame importance scores from the rollout are mapped to real timestamps using the feature encoder's stride. Wav2Vec2's feature encoder has a stride of 320 samples (20ms at 16 kHz). Each frame index  $i$  corresponds to time interval  $[i * 320 / 16000, (i+1) * 320 / 16000]$  seconds. The scores are min-max normalized to  $[0, 1]$ .

#### Temporal Mapping

```
def map_attention_to_timeline(layer_maps):
    stride = 320 # feature encoder stride in samples
    sr = 16000
    aggregate = layer_maps[-1] # rollout importance
    aggregate = (aggregate - min) / (max - min + 1e-8) # normalize
    timeline = []
    for i in range(n_frames):
        timeline.append({
            "frame": i,
            "start_ms": i * stride / sr * 1000,
            "end_ms": (i+1) * stride / sr * 1000,
            "attention_score": float(aggregate[i]),
```

```
    })  
    return timeline
```

## 7.4 Heatmap Visualization

The visualizer (`app/xai/visualizer.py`) uses `matplotlib` to render the timeline scores as a filled line chart (crimson fill, darkred line). The plot is saved to a PNG in-memory buffer, then base64-encoded for transport in the JSON response and embedding in the PDF report.

## 7.5 XAI Output Fields

**overall\_heatmap:** Base64-encoded PNG of the attention vs time chart

**layer\_count:** Number of transformer layers (always 12 for W2V2 base)

**timeline:** Array of {frame, start\_ms, end\_ms, attention\_score} objects

## 8. PDF Report Generator

### 8.1 Report Structure

The PDF report is generated using fpdf2 (FPDF). Each report contains four major sections:

Figure 6: PDF Report Layout

```
+=====+
|  SYSTEM VERDICT BLOCK                               |
|  (Red=SP00F or Green=B0NA-FIDE full-width bar) |
+=====+
|  1. Decision Executive Summary                       |
|  Table: Attribute | Value | Interpretation |
+-----+
|  2. Multi-Model Signal Breakdown                     |
|  AASIST: score, focus, result                       |
|  Wav2Vec2: score, focus, result                     |
|  Ensemble: score, focus, result                     |
+-----+
|  3. Visual Explainability (XAI)                      |
|  Heatmap image + temporal analysis text              |
+-----+
|  4. Calibration Guide                                |
|  Confidence bar 0.0---[marker]---1.0                |
|  Interpretation policy note                          |
+=====+
```

### 8.2 Generation Details

The report is generated on-the-fly after each detection call. The generate\_report() function in app/utils/pdf\_report.py receives the request\_id, prediction, confidence, scores, process\_time\_ms, and attention\_maps dict. It constructs a ReportPDF object, draws each section using fpdf2 primitives, and returns the PDF as bytes.

### 8.3 Key Implementation Notes

Latin-1 limitation: fpdf2 Helvetica font does not support Unicode. All special characters (em-dashes, smart quotes) are replaced with ASCII equivalents.

Table wrapping: A custom \_multi\_cell\_row helper uses multi\_cell for text wrapping inside table cells, unlike fixed-height cell() which truncates.

Verdict block: Full-width colored rectangle (red #DC3232 or green #28B450) with white text, drawn at the top of the report body.

Calibration bar: A dual-colored horizontal bar (green left half, red right half) with a black triangular marker at the confidence position.

In-memory cache: PDF bytes stored in the global REPORTS dict keyed by request\_id UUID. Retrieved on GET /api/v1/report/{id}.

## 9. API Layer & FastAPI Application

### 9.1 Application Entry Point

#### FastAPI Application Setup

```
# app/main.py
app = FastAPI(title="Audio Deepfake Detection API",
              version="1.0.0",
              lifespan=lifespan)
app.add_middleware(CORSMiddleware,
                  allow_origins=["*"],
                  allow_methods=["*"],
                  allow_headers=["*"])
app.include_router(router)
```

### 9.2 Lifespan & Startup

The lifespan handler (`app/api/lifespan.py`) runs on startup: it checks ffmpeg availability, then initializes and loads all three models asynchronously via `init_components()`. On shutdown, it unloads the models to free memory. The startup timestamp is captured for uptime calculation in the health endpoint.

### 9.3 Route Map

#### Figure 7: API Endpoints

```
GET /api/v1/health
Returns: JSON with model load status, device, uptime

POST /api/v1/detect
Body: multipart/form-data with 'file' field (UploadFile)
Returns: JSON with prediction, confidence, scores, XAI maps, report link

GET /api/v1/report/{request_id}
Returns: application/pdf binary download
```

### 9.4 Request Lifecycle (Detailed)

#### Figure 8: Request Lifecycle

1. HTTP Request arrives at FastAPI router
2. CORS middleware checks origin (wildcard: any origin allowed)
3. File extension validated against `supported_extensions` set
4. File size validated against `max_upload_size_mb` (50 MB)
5. File bytes read into memory
6. `InferenceOrchestrator.detect()` called (always `xai_enabled=True`)
7. UUID `request_id` generated
8. Response schema populated (`DetectResponse` or `XaiResponse`)
9. PDF report generated via `generate_report()`
10. PDF bytes stored in `REPORTS` dict with `request_id` key
11. JSON response returned with `report_download_link` field

### 9.5 Response Schemas

#### Pydantic Response Models

```
class DetectResponse(BaseModel):
    request_id: str
    prediction: str          # "spoof" or "bona-fide"
    confidence: float        # 0.0 to 1.0
    scores: dict[str, float] # wav2vec2, aasist, ensemble
    process_time_ms: float

class XaiAttentionMap(BaseModel):
    overall_heatmap: str     # base64 PNG
```

```
    layer_count: int
    timeline: list[dict]      # {start_ms, end_ms, attention_score}

class XaiResponse(DetectResponse):
    attention_maps: XaiAttentionMap | None = None

class HealthResponse(BaseModel):
    status: str                # "ok"
    wav2vec2_loaded: bool
    aassist_loaded: bool
    meta_classifier_loaded: bool
    device: str                # "cpu"
    uptime_seconds: float
```

## 10. Dependency Injection & Component Lifecycle

### 10.1 AppComponents

The AppComponents class (app/dependencies.py) acts as a service container that holds singleton references to all three model pipelines. It is initialized once at startup via `init_components()` and shared across requests through FastAPI's Depends mechanism.

#### Dependency Container

```
class AppComponents:
    def __init__(self):
        self.wav2vec2 = Wav2Vec2Pipeline()
        self.aasist = AASISTPipeline()
        self.meta_classifier = MetaClassifier(settings.meta_classifier_path)

    @property
    def orchestrator(self) -> InferenceOrchestrator:
        return InferenceOrchestrator(self.wav2vec2, self.aasist, self.meta_classifier)

    async def init_components() -> AppComponents:
        comps = AppComponents()
        await comps.wav2vec2.load()          # downloads from HF Hub
        await comps.aasist.load()            # loads from weights/aasist_backbone.pt
        comps.meta_classifier.load()         # loads from weights/meta_mlp.pt
        return comps

    async def get_orchestrator() -> InferenceOrchestrator:
        comps = await init_components()     # returns cached singleton
        return comps.orchestrator
```

### 10.2 Singleton Pattern

The `_components` module-level variable caches the AppComponents instance after first initialization. Subsequent calls to `init_components()` return the same instance, ensuring models are loaded exactly once. FastAPI routes receive the orchestrator via `Depends(get_orchestrator)`, which is resolved per-request but returns the shared instance.

## 11. Configuration System

### 11.1 Settings (Pydantic BaseSettings)

All configuration is managed through a Pydantic BaseSettings class (app/config.py). Settings can be overridden via environment variables with the ADFD\_ prefix.

#### Settings Class

```
class Settings(BaseSettings):
    sample_rate: int = 16000
    max_audio_seconds: int = 300
    max_audio_samples: int = sample_rate * max_audio_seconds

    aasist_nb_samp: int = 64600
    device: str = "cpu"
    weights_dir: Path = Path("weights")

    wav2vec2_model_id: str = "Vansh180/deepfake-audio-wav2vec2"
    aasist_pytorch_path: Path = weights_dir / "aasist_backbone.pt"
    aasist_download_url: str = "https://github.com/clovaai/aasist/raw/main/..."
    meta_classifier_path: Path = weights_dir / "meta_mlp.pt"

    xai_layers_to_visualize: int = 4
    api_host: str = "0.0.0.0"
    api_port: int = 8000
```



```
api_workers: int = 1
max_upload_size_mb: int = 50
log_level: str = "INFO"
model_config = {"env_prefix": "ADFD_"}

@property
def supported_extensions(self) -> set[str]:
    return {".wav", ".flac", ".mp3", ".m4a", ".ogg"}
```

## 12. Deployment Architecture

### 12.1 Docker Container

Figure 9: Docker Container Structure

```

+=====+
| Docker Image: audio-deepfake-detector:latest |
| Azure ACR: deepfakeprojectacr2026.azurecr.io |
+=====+
| Base: python:3.11-slim |
| +-- PyTorch CPU (--index-url cpu) |
| +-- ffmpeg (for MP3/M4A/OGG decoding) |
| +-- curl (healthcheck) |
| +-- pip install -r requirements.txt |
| +-- COPY app/ weights/ docker/ |
| EXPOSE 8000 |
| HEALTHCHECK curl -f http://localhost:8000/health |
| CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"] |
+=====+

```

### 12.2 Azure Deployment

Figure 10: Azure Architecture

```

Internet
|
v
Azure Container Apps (francecentral)
|
+-- Container: audio-deepfake-detector
|   - CPU-only (no GPU)
|   - 4 GB memory limit
|   - Port 8000
|   - Ingress: external, HTTPS
|
+-- ACR: deepfakeprojectacr2026.azurecr.io
|   - Image: audio-deepfake-detector:latest
|
+-- Health probe: GET /api/v1/health

URL: https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io

```

### 12.3 Dockerfile Highlights

The Docker build uses a single-stage `python:3.11-slim` image. The critical optimization is the `--index-url https://download.pytorch.org/whl/cpu` flag when installing PyTorch, which downloads the CPU-only wheel (~120 MB) instead of the CUDA bundle (~2+ GB). This reduces the build time on Azure ACR from 20+ minutes to approximately 5 minutes and keeps the final image below 1.5 GB.

#### Dockerfile

```

# docker/Dockerfile (key lines)
FROM python:3.11-slim
RUN apt-get update && apt-get install -y ffmpeg curl
COPY requirements.txt .
RUN pip install --no-cache-dir --index-url \
    https://download.pytorch.org/whl/cpu -r requirements.txt
COPY app/ app/
COPY weights/ weights/
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s \
    CMD curl -f http://localhost:8000/api/v1/health || exit 1
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

## 12.4 Docker Compose (Local Dev)

### Docker Compose

```
# docker/docker-compose.yml
services:
  audio-deepfake-detector:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    ports:
      - "8000:8000"
    volumes:
      - ../weights:/weights
    mem_limit: 4g
    healthcheck:
      test: curl -f http://localhost:8000/api/v1/health || exit 1
      interval: 30s
      timeout: 10s
```

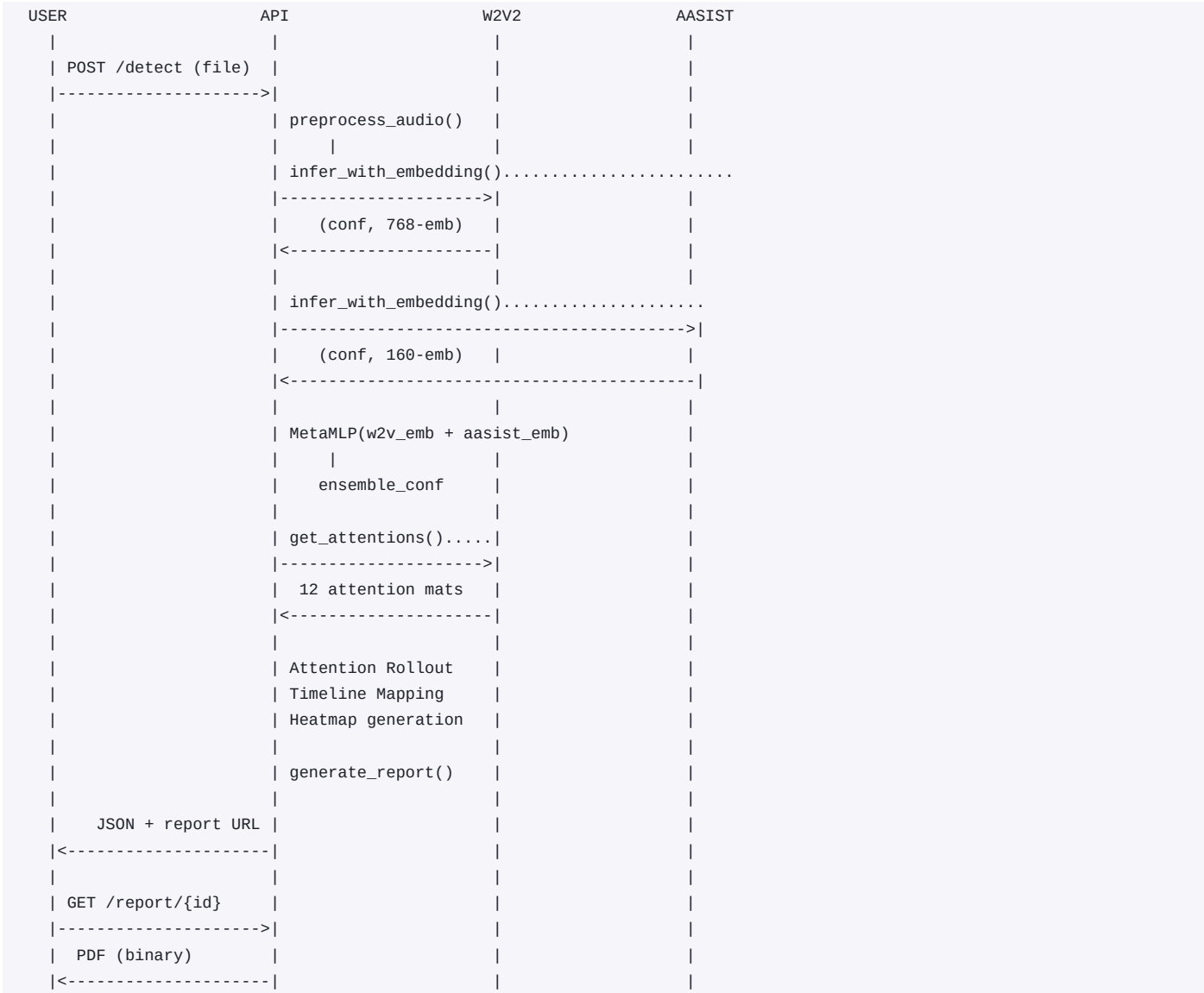
## 13. Project Directory Structure

Figure 11: File Tree

```
C:\Users\DELL\Desktop\Audio\
|
+-- app/
|   +-- __init__.py
|   +-- main.py           FastAPI app entry
|   +-- config.py         Pydantic settings
|   +-- dependencies.py    DI container
|   |
|   +-- api/
|       +-- routes.py      3 endpoints (health, detect, report)
|       +-- schemas.py     Pydantic request/response models
|       +-- lifespan.py    Startup/shutdown handler
|       |
|   +-- models/
|       +-- wav2vec2_pipeline.py  W2V2 loader + inference
|       +-- aasist_pipeline.py    AASIST model + inference
|       +-- meta_classifier.py    MetaMLP + MetaClassifier
|       |
|   +-- inference/
|       +-- orchestrator.py      Coordinates multi-model pipeline
|       +-- preprocessor.py      Audio bytes -> numpy array
|       |
|   +-- xai/
|       +-- attention_capturer.py Attention Rollout algorithm
|       +-- temporal_mapper.py    Frame scores -> timestamps
|       +-- visualizer.py         matplotlib -> base64 PNG
|       |
|   +-- utils/
|       +-- audio_utils.py        ffmpeg pipe, load, normalize
|       +-- pdf_report.py         fpdf2 report generator
|       |
|   +-- weights/
|       +-- aasist_backbone.pt     AASIST pretrained weights
|       +-- meta_mlp.pt           Dual-branch MLP trained weights
|       |
|   +-- docker/
|       +-- Dockerfile             CPU-only PyTorch, ffmpeg
|       +-- docker-compose.yml     Local dev setup
|       |
|   +-- research_and_training/
|       +-- kaggle_mlp_trainer.py  MetaMLP training notebook
|       |
|   +-- requirements.txt           Python dependencies
|   +-- .dockerignore             Excludes venv, notebooks, tests
|   +-- .gitignore
```

# 14. Complete Data Flow Sequence Diagram

Figure 12: Full Sequence Diagram



## 15. Meta-Classifer Training Pipeline

### 15.1 Training Overview

The dual-branch MetaMLP is trained separately from the base models. The process involves extracting embeddings from W2V2 and AASIST for the ASVspoof 2019 LA dataset, then training the lightweight MLP (only ~200K parameters) to fuse them optimally.

### 15.2 Training Steps

Step 1 -- Feature Extraction: For each audio file in the ASVspoof 2019 LA training set, run `W2V2.infer_with_embedding()` and `AASIST.infer_with_embedding()` to obtain the 768-dim and 160-dim embeddings. Save these as a feature cache (numpy arrays) along with labels.

Step 2 -- Train/Val Split: Split the cached features into 80% training and 20% validation sets, stratified by label to maintain class balance.

Step 3 -- MLP Training: Train the MetaMLP with Binary Cross-Entropy loss (`BCEWithLogitsLoss`). Use AdamW optimizer with `lr=1e-4` and `weight_decay=1e-5`. Batch size is 64. Train for 50 epochs with early stopping (`patience=5`) based on validation accuracy.

Step 4 -- Evaluation: The final model achieves 99.17% validation accuracy on the held-out 20% of ASVspoof 2019 LA. Weights are saved to `weights/meta_mlp.pt` (~488 KB).

### 15.3 Training code location

Script: [research\\_and\\_training/kaggle\\_mlp\\_trainer.py](#)

## 16. Performance & Resource Usage

### 16.1 CPU Inference Latency

Wav2Vec2 forward pass: ~400-800 ms for 5s audio  
AASIST forward pass: ~100-200 ms for 4s audio  
Meta-classifier forward pass: <1 ms  
XAI attention processing: ~50-100 ms  
PDF generation: ~200-400 ms  
Audio preprocessing + ffmpeg decode: ~50-200 ms  
Total typical: 800-2000 ms (5s audio on CPU)

### 16.2 Memory Usage

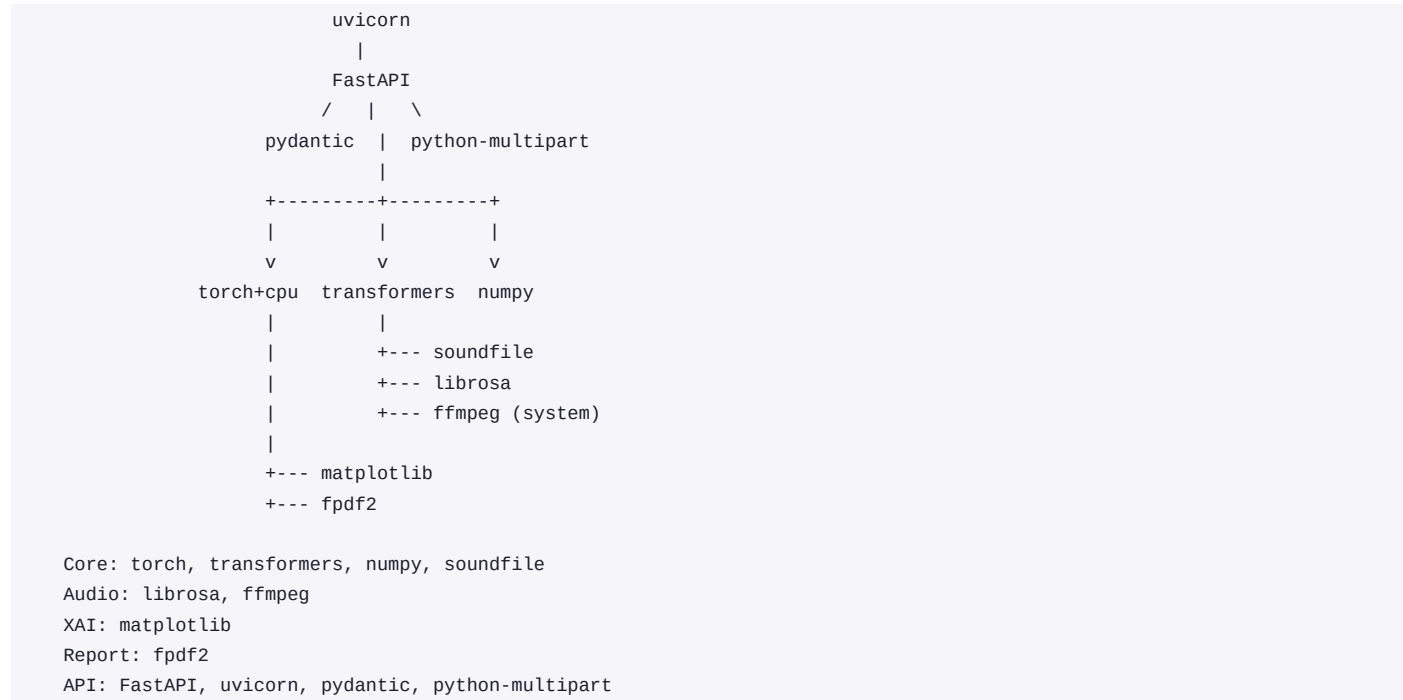
Wav2Vec2 model: ~450 MB (transformer + feature encoder)  
AASIST model: ~50 MB  
MetaMLP model: ~2 MB  
Python runtime + FastAPI: ~100 MB  
Total (steady state): ~600-700 MB  
Docker memory limit: 4 GB (provides headroom for requests)

### 16.3 Model Size

Wav2Vec2: ~450 MB (downloaded from HF Hub on first load)  
AASIST: ~80 MB (weights/aasist\_backbone.pt)  
MetaMLP: ~488 KB (weights/meta\_mlp.pt)

# 17. Dependency Graph

Figure 13: Dependency Graph





18. Complete File Reference

|     |                                 |                                                                                   |               |
|-----|---------------------------------|-----------------------------------------------------------------------------------|---------------|
| 1.  | app/main.py                     | FastAPI app creation, CORS middleware, router inclusion                           | (22 lines)    |
| 2.  | app/config.py                   | Pydantic BaseSettings with all configurable parameters                            | (48 lines)    |
| 3.  | app/dependencies.py             | AppComponents container, singleton init, FastAPI Depends factory                  | (38 lines)    |
| 4.  | app/api/routes.py               | 3 endpoints: GET /health, POST /detect, GET /report/{id}                          | (96 lines)    |
| 5.  | app/api/schemas.py              | Pydantic models: DetectResponse, XaiResponse, XaiAttentionMap, HealthResponse     | (41 lines)    |
| 6.  | app/api/lifespan.py             | Startup: ffmpeg check, model loading. Shutdown: model unloading                   | (24 lines)    |
| 7.  | app/models/wav2vec2_pipeline.py | W2V2: load, preprocess, infer, infer_with_embedding, get_attentions               | (86 lines)    |
| 8.  | app/models/aasist_pipeline.py   | AASIST: SincConv, GAT layers, AASISTModel, pipeline inference                     | (320 lines)   |
| 9.  | app/models/meta_classifier.py   | MetaMLP (dual-branch), MetaClassifier wrapper with fallback                       | (99 lines)    |
| 10. | app/inference/orchestrator.py   | InferenceOrchestrator: calls all 3 models, XAI, returns DetectionResult/XaiResult | (17 lines)    |
| 11. | app/inference/preprocessor.py   | Short wrapper: bytes -> numpy via audio_utils + preprocess_pipeline               | (10 lines)    |
| 12. | app/xai/attention_capturer.py   | Attention Rollout algorithm, per-layer aggregation                                | (36 lines)    |
| 13. | app/xai/temporal_mapper.py      | Frame-level attention -> timestamped timeline dicts                               | (32 lines)    |
| 14. | app/xai/visualizer.py           | Matplotlib: timeline line chart -> base64 PNG heatmap                             | (61 lines)    |
| 15. | app/utils/audio_utils.py        | ffmpeg pipe, soundfile load, resample, peak normalize, truncate                   | (68 lines)    |
| 16. | app/utils/pdf_report.py         | fpdf2 report: verdict block, summary table, model cards, XAI, calibration bar     | (104 lines)   |
| 17. | docker/Dockerfile               | Multi-stage: python:3.11-slim, CPU-only PyTorch, ffmpeg, curl                     | (~30 lines)   |
| 18. | docker/docker-compose.yml       | Local dev: port 8000, weights volume, 4GB limit, healthcheck                      | (~20 lines)   |
| 19. | requirements.txt                | All Python dependencies                                                           | (~20 lines)   |
| 20. | .dockerignore                   | Excludes venv, notebooks, tests, .git                                             | (~10 lines)   |
| 21. | generate_frontend_guide.py      | Generates frontend_integration_guide.pdf for frontend devs                        | (706 lines)   |
| 22. | generate_architecture_guide.py  | Generates this architecture document                                              | (~1000 lines) |

Total: 22 files | ~2,300 lines of Python | Last updated: June 2026